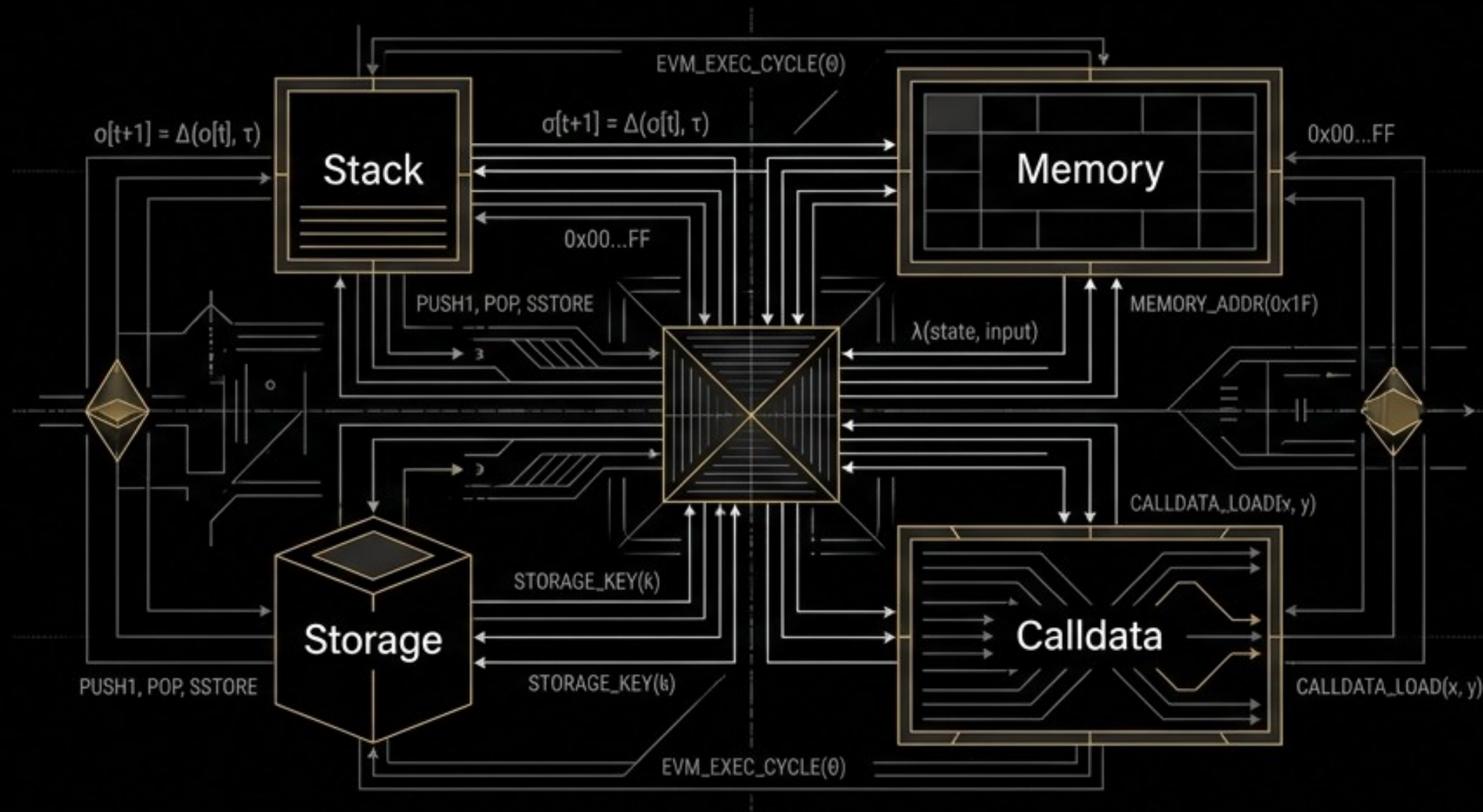


The Ethereum Virtual Machine (EVM) Architecture

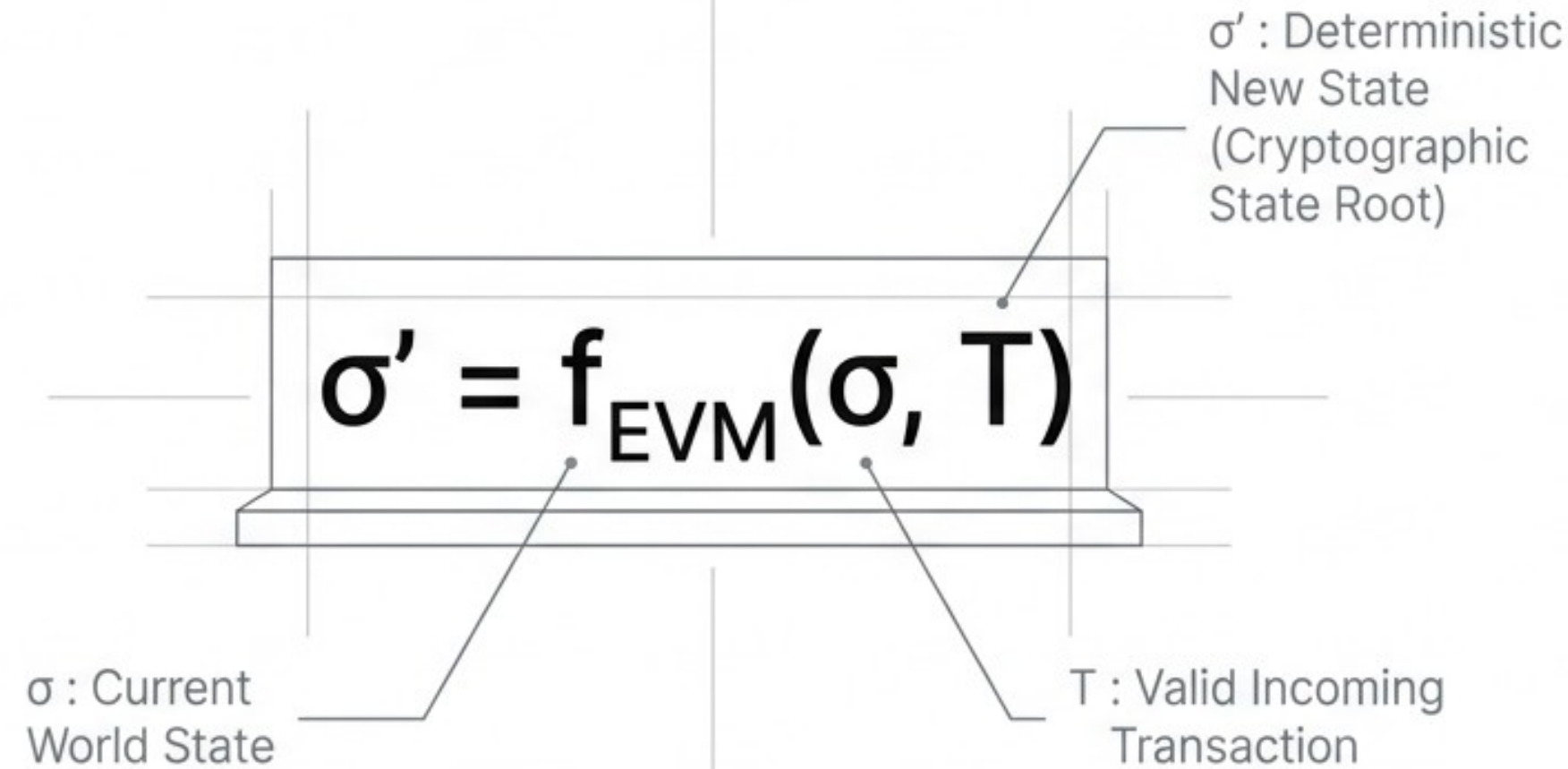
Anatomy, Execution Cycles, and Inter-Contract Mechanics

A deep-dive technical masterclass into the fundamental internal mechanics of the EVM runtime environment.

Focus areas: Deterministic state transitions, the 6-part memory taxonomy, opcode fetch-decode cycles, and cross-contract executions.



The Deterministic State Engine



Absolute Determinism

Tens of thousands of independent global validators execute identical bytecode on varying hardware.

Zero-Tolerance Consensus

Every node must arrive at a bit-for-bit identical cryptographic state root. A single differing bit results in immediate consensus isolation.

Taxonomy of EVM Data Regions

The EVM is a quasi-Turing-complete, stack-based machine.

Volatile Context (Erased Post-Transaction)

Stack: 1,024 slots (256-bit words) for LIFO computation.

Memory: Linear, byte-addressable ephemeral scratchpad.

Calldata: Read-only transaction payload from the caller.

ReturnData: Output buffer from sub-contract calls.

Persistent State (Stored to Validator SSDs)

Storage: High-cost 2^{256} key-value database bound to the contract account.

Code: Immutable ROM containing the contract's bytecode.

Volatile Compute Space

1

The Stack (LIFO Engine)

Architecture: Operates on a Last-In, First-Out (LIFO) model, not hardware registers (x86/ARM).

Specs: Exactly 1,024 item capacity. Word size is strictly 256 bits (32 bytes) to optimize for Keccak-256 and secp256k1 cryptography.

Fatal Exception (Stack too deep): SWAP and DUP opcodes only reach the top 16 slots. Exceeding 16 simultaneous local variables causes compilation failure.

2

Volatile Memory (Linear Scratchpad)

Architecture: Dynamically allocated, byte-addressable linear RAM (MLOAD/MSTORE). Erased completely upon transaction halt.

Defense Mechanism: Features a Quadratic Expansion Gas formula to prevent validator RAM exhaustion.

$$C_{\text{mem}}(a) = 3 \times a + \left\lfloor \frac{a^2}{512} \right\rfloor$$

(where a = 32-byte words allocated)

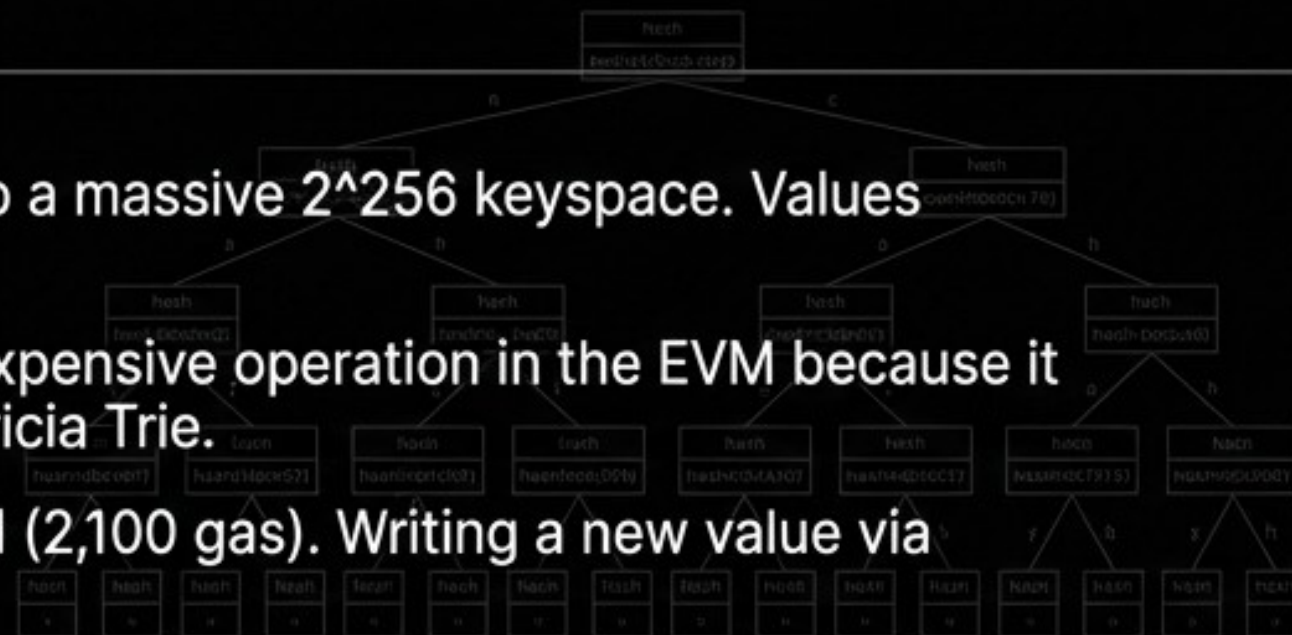
Persistent State & Payload

Storage (The Permanent Database)

Architecture: Isolated key-value database mapped to a massive 2^{256} keyspace. Values default to 0x00.

Disk I/O Economics: Modifying storage is the most expensive operation in the EVM because it forces global validator SSD writes via the Merkle Patricia Trie.

Gas Penalties: SLOAD warm (100 gas) vs SLOAD cold (2,100 gas). Writing a new value via SSTORE costs a massive 20,000 gas.



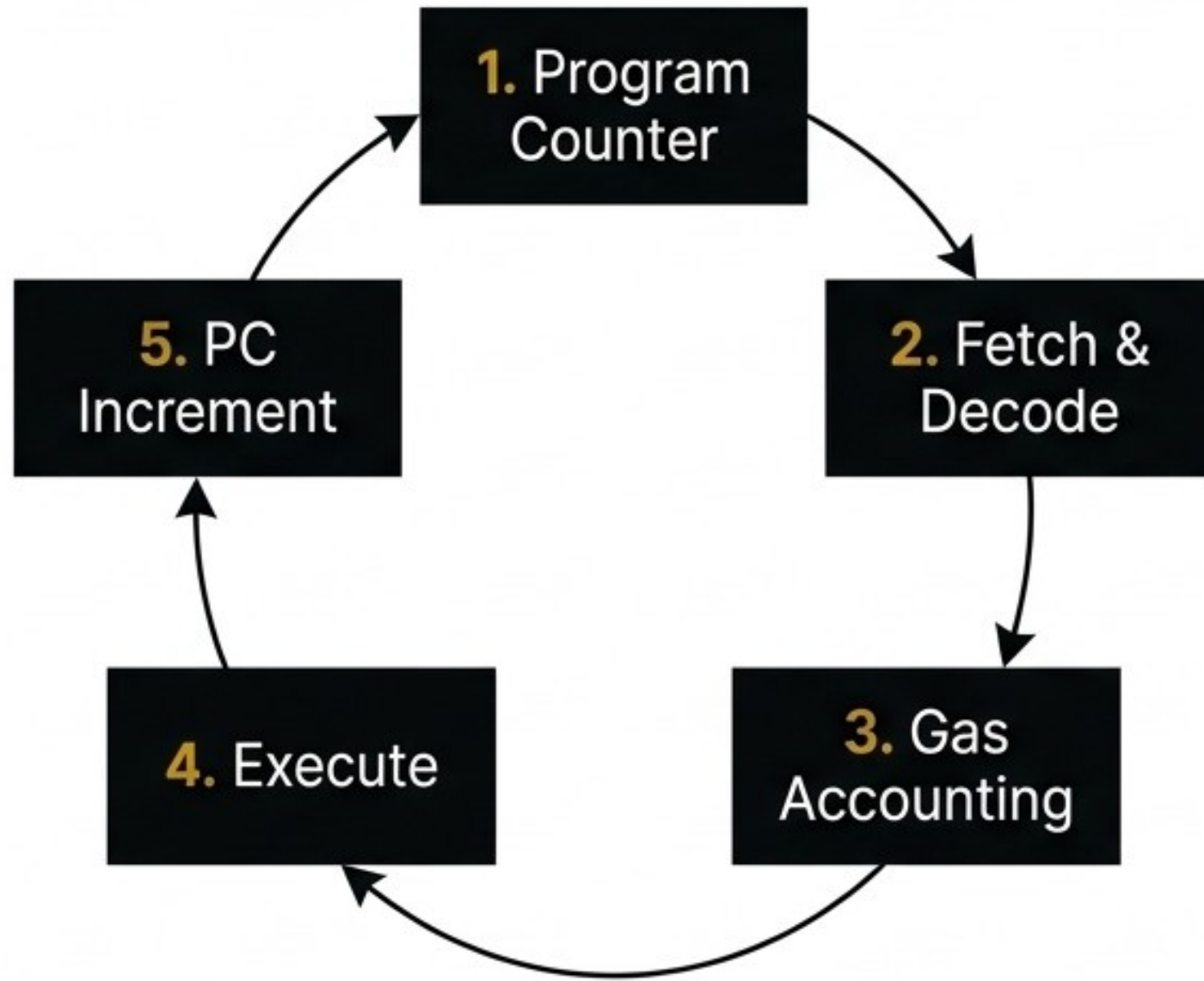
Calldata (Read-Only Payload)

Architecture: Immutable byte array containing execution instructions and ABI arguments.

Structure: First 4 bytes = Function Selector (e.g., keccak256("transfer(address,uint256)") [0..4]). Subsequent 32-byte chunks = Arguments.

Strategic Utility: Extremely gas-efficient. Acts as the backbone for L2 Rollups to post compressed transaction batches to Layer 1 cheaply.

The Low-Level Execution Cycle



1. Program Counter (PC):

Internal pointer targeting the next bytecode index.

2. Fetch & Decode:

Reads the byte at the PC and matches it to the opcode table (e.g., `0x01 = ADD`, `0x54 = SLOAD`).

3. Gas Accounting:

Deducts static/dynamic gas. Insufficient gas triggers an immediate abort.

4. Execute: Pulls operands from the Stack, mutates Memory/Storage, and pushes the result back.

5. PC Increment: Advances linearly until hitting STOP, RETURN, or REVERT.

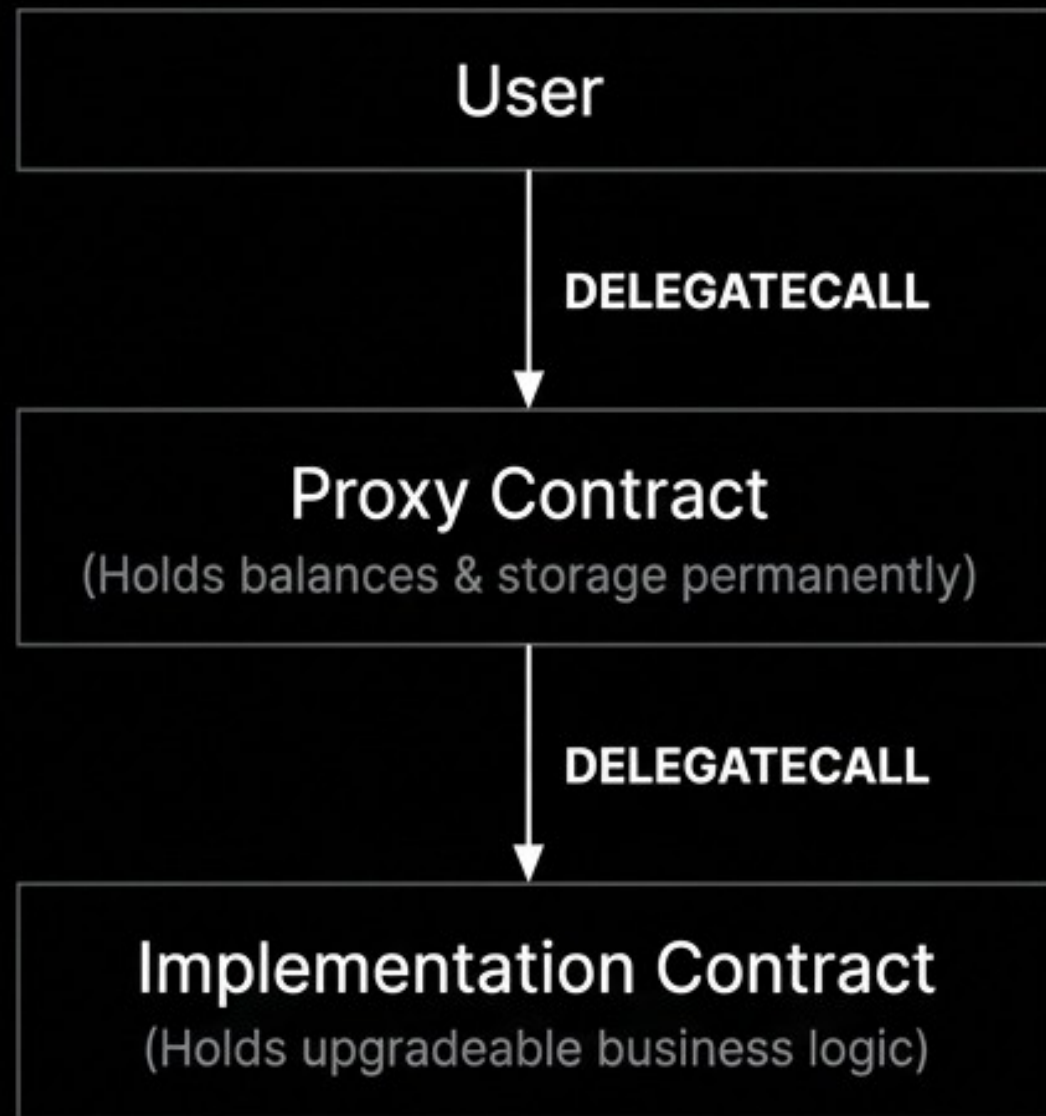
Inter-Contract Execution Matrix

Composability and the mechanics of shifting execution contexts.

	CALL	DELEGATECALL
Mechanism	Contract A executes a function inside Contract B's environment.	Contract A borrows Contract B's code and runs it in Contract A's environment.
Identity (msg.sender)	Changes to Contract A.	Persists as the original caller.
Storage Target	Mutates Contract B's database.	Mutates Contract A's database.

Note: DELEGATECALL is the foundational opcode that makes upgradeable smart contracts possible.

The Proxy Pattern & The Parity Freeze



Upgrades are achieved by merely changing the implementation pointer in the Proxy, safeguarding user funds.

The Flaw: Hundreds of multi-sig wallets delegated their logic to a single shared library contract that was left uninitialized.

The Trigger: A user accidentally claimed ownership of the library and executed the SELFDESTRUCT opcode.

The Impact: The library's bytecode was annihilated, permanently bricking all dependent proxies. **513,774 ETH** across **587** wallets were frozen forever without recourse.

Deterministic Deployment Models

CREATE (Nonce-Dependent)

$$\text{Address} = \text{Rightmost20Bytes}(\text{Keccak-256}(\text{RLP}(\text{Sender}, \text{Nonce})))$$

Mechanism: The resulting 20-byte address is strictly tied to the chronological transaction sequence (nonce) of the deployer's account. Highly dynamic.

CREATE2 (Counterfactual Determinism)

$$\text{Address} = \text{Rightmost20Bytes}(\text{Keccak-256}(0\text{xff} \parallel \text{Sender} \parallel \text{Salt} \parallel \text{Keccak-256}(\text{InitCode})))$$

Mechanism: Computes exact contract addresses before deployment.

Key Variables: The **0xff** prefix prevents collisions with CREATE. Relies on a random **Salt** and the **InitCode bytecode** hash.

Impact: Enables Uniswap V2/V3 to mathematically deduce liquidity pool addresses instantly without paying gas for on-chain registry lookups.

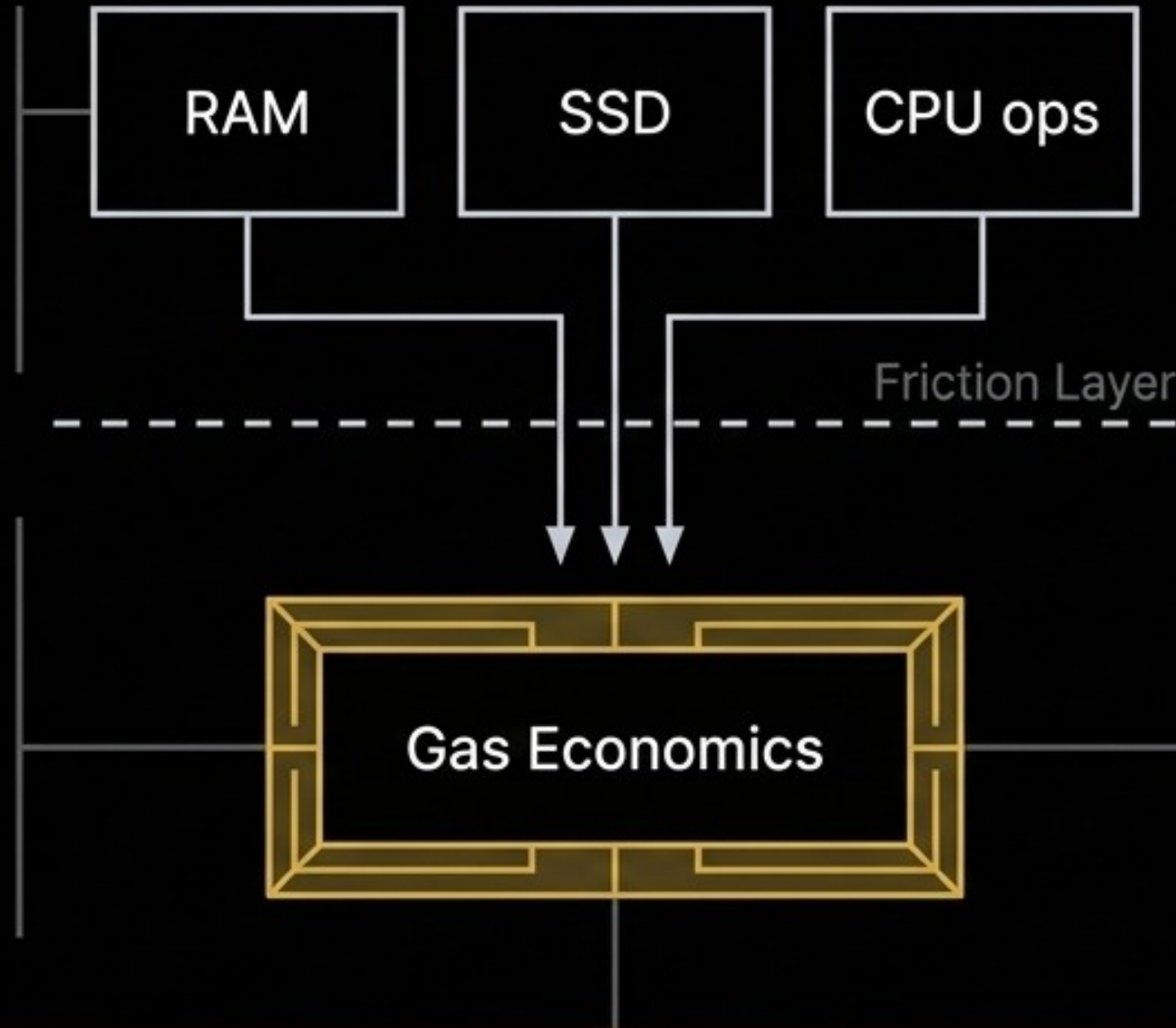
The Bridge to Execution Economics

The Architectural Reality

We have mapped the physical limits of the Stack, the quadratic constraints of Memory, and the extreme I/O cost of Persistent Storage.

The Engineering Question

How does a decentralized protocol establish a fair, trustless price for every low-level opcode? Why is reading cold disk data thousands of times more expensive than LIFO computation?



The Economic Solution

The EVM cannot survive without its economic physics.

Next Module: Gas Economics, Execution Halting, and EIP-1559

How the protocol prices computation, halts infinite loops, and burns base fees to stabilize the world computer.